

# Tectopia

Seguridad en una infraestructura k3s en producción

Benjamín Saavedra · Joaquín Veloz

IIC2531 - Seguridad Computacional - 2026-1

# Mapa rápido por clase

| Clase | Tema                                                    | Dónde lo van a ver hoy     |
|-------|---------------------------------------------------------|----------------------------|
| 01    | Privilegios mínimos, modelo de amenaza, riesgo residual | Casi toda la presentación  |
| 02    | OKWS, separación de privilegios                         | Guardia, RBAC, Postgres    |
| 03/04 | Namespaces, default deny                                | Aislamiento de red         |
| 05    | Authenticate→Authorize→Audit                            | Modelo de guardia          |
| 14/15 | TLS, PKI                                                | Cloudflare, Sealed Secrets |
| 16/18 | XSS, CSRF, OWASP Top 10                                 | Web security               |
| 22    | Cadena de suministro                                    | CVEs, Trivy                |

# Showcase rápido del proyecto!

- Si! Esta vivo en producción!

<https://tectopia.cl>

# Qué hicimos

6 medidas prometidas sobre k3s (4 VMs, producción real). Hoy recorreremos los mecanismos técnicos concretos detrás de cada una:

1. Separación de privilegios (namespaces/pods/RBAC)
2. Detección de CVEs (cadena de suministro)
3. Autenticación de Redis + ACL
4. Defensa web (XSS, MIME-sniffing, clickjacking)
5. Cloudflare Zero Trust + TLS
6. Modelo de guardia (Auth → Authz → Audit)

## Modelo de amenaza (1/2): objetivo y atacantes

**Objetivo, en orden de prioridad: Integridad > Confidencialidad > Disponibilidad.**

La economía y los resultados de partidas pierden valor si se pueden falsificar. Los datos de usuario no son tan sensibles como en un sistema bancario o médico. Una caída temporal es molesta, no grave.

**Atacantes considerados:**

1. Usuario con cuenta válida que intenta hacer trampa o atacar a otros jugadores
2. Atacante externo que explota vulnerabilidades del servidor o de dependencias
3. Atacante de cadena de suministro

**Fuera de alcance:** un atacante ya dentro de la LAN privada, y el compromiso físico de las VMs.

## Modelo de amenaza (2/2): política y mecanismo

**Política (algunas reglas):** solo usuarios autenticados pueden jugar o chatear. Solo administradores pueden banear o asignar roles. Un moderador nunca puede moderar a un administrador. Toda acción administrativa queda registrada. Los secretos de infraestructura nunca se guardan en texto plano.

| Política                           | Mecanismo                               |
|------------------------------------|-----------------------------------------|
| Solo autenticados                  | JWT de Clerk verificado en cada request |
| Solo admins banear / asignan roles | RBAC de aplicación, permisos por rol    |
| Mod no modera a admin              | <code>canModerate()</code> , regla fija |
| Acción admin registrada            | Audit log en dos niveles                |
| Secretos nunca en texto plano      | Sealed Secrets                          |

## Arquitectura: 3 tiers

| Tier        | Namespace                                                 | Postgres/Redis      |
|-------------|-----------------------------------------------------------|---------------------|
| Crítico     | <code>buscaviejos-main</code>                             | dedicado            |
| Persistente | <code>buscaviejos-develop</code> + <code>release/*</code> | compartido          |
| Preview     | <code>preview-{sha7}</code>                               | compartido, efímero |

## Aislamiento de sistema y red

- **Namespaces de Linux** (PID, mount, net) vía contenedores administrados por Kubernetes: un compromiso en la app no toca el host
- **NetworkPolicy = Default Deny**: todo el egress está prohibido salvo lo explícitamente permitido

```
egress:
- to: [podSelector: {}]                # intra-namespace
- to: [namespaceSelector: kube-system]
  ports: [{port: 53}]                  # DNS
- ports: [{port: 443}]                 # Clerk JWKS – api.clerk.com
```



## RBAC de Kubernetes

El runner de CD **sí tiene** RBAC acotado para su propio control plane:

|                                               |                         |
|-----------------------------------------------|-------------------------|
| Role/buscaviejos-cd-runner-set-gha-rs-manager | (namespace arc-system)  |
| Role/buscaviejos-cd-runner-set-...-listener   | (namespace arc-runners) |

La brecha está en permisos de infraestructura que todavía son demasiado amplios y conviene acotar por componente.

# RBAC dentro del juego mismo

```
type Role = "user" | "mod" | "admin"  
const RoleHierarchy = { user: 1, mod: 2, admin: 3 }
```

| Rol   | Permisos (extracto de 17 totales)                       |
|-------|---------------------------------------------------------|
| user  | chat, jugar, agregar amigos, editar perfil              |
| mod   | + ver detalles de usuario, moderar mensajes, restringir |
| admin | + banear, asignar roles, gestionar economía             |

```
canModerate(actor, target) {  
  if (target.role === "admin") return false  
  return RoleHierarchy[actor] > RoleHierarchy[target]  
}
```

# Autenticación criptográfica asimétrica

El backend no confía en el cliente, verifica la firma del JWT matemáticamente:

```
// REST – requireAuth.ts
const { userId } = getAuth(req)           // valida JWT vía Clerk

// WebSocket – sockets/index.ts:44
const { sub: clerkUserId } = await verifyToken(token, { ... })
```

Verificación contra llaves públicas (JWKS): el mismo control en HTTP y en el handshake de WebSocket, no solo en un punto de entrada.

# El guardia

- Tenemos un punto único de decisión de autenticación:

```
app.set("trust proxy", 1) // app.ts:13, confía solo en el salto directo
```

- Todo el tráfico choca contra `requireAuth.ts`, después `requireUser.ts`, después `requirePermission.ts`: la lógica de acceso nunca se dispersa
- `trust proxy` evita que un atacante falsifique su IP vía cabeceras HTTP para ensuciar el log de auditoría (mitiga *IP spoofing* sobre el guardia)

## El guardia audita cada decisión

```
// auditLog.ts, 5 líneas, deliberadamente mínimo
export const audit = (event, meta = {}) =>
  process.stdout.write(JSON.stringify({ ts: new Date().toISOString(),
    level: "audit", event, ...meta }) + "\n")
```

`requirePermission.ts` llama `audit("authz.granted" | "authz.denied", ...)`  
como punto único de paso: ninguna ruta admin puede saltarse el registro.

## Ejemplo: Auditoría real

```
{"level": "Metadata", "verb": "watch",  
  "user": {"username": "system:serviceaccount:monitoring:...grafana"},  
  "sourceIPs": ["10.42.0.82"], "objectRef": {"resource": "secrets", ...}}
```

Auditoría a dos niveles: aplicación (Loki/Grafana) y API server de k3s.

Cualquier acceso a `secrets` en el clúster queda registrado: quién, desde dónde, cuándo.

# Mapa completo: guardia a recurso

- Lugares donde podemos mejorar...

| Recurso               | Autentica                          | Autoriza                                 | Audita                             |
|-----------------------|------------------------------------|------------------------------------------|------------------------------------|
| Endpoints HTTP        | JWT de Clerk                       | Middleware en cascada                    | <code>authz.*</code> -> Loki       |
| WebSocket             | Token en el handshake              | Pertenencia a canal                      | —                                  |
| Postgres              | <code>scram-sha-256</code> por rol | DML vs. DDL (solo en <code>main</code> ) | k3s audit ( <code>secrets</code> ) |
| Redis                 | Usuario+contraseña ACL             | Comandos por categoría                   | —                                  |
| Clúster (kubectl/API) | ServiceAccount/SSH                 | RBAC (ver <code>cluster-admin</code> )   | k3s audit policy                   |
| Internet              | —                                  | Cloudflare Tunnel (solo enruta)          | Logs de Cloudflare                 |

## Privilegios en Postgres (1/2)

En el tier persistente ( `develop` y todos los `release/*` ), el mismo rol que usa el backend es dueño del esquema:

```
$ DROP TABLE users    -- como buscaviejos_develop
ERROR: cannot drop table users because other objects depend on it
HINT: Use DROP ... CASCADE
```

**No es un error de permisos, el rol sí podría hacerlo.**



## Privilegios en Postgres (2/2)

buscaviejos-main: el backend se conecta como svc\_app, no como dueño del esquema.

```
$ DROP TABLE users -- como svc_app  
ERROR: must be owner of table users
```

Migraciones (DDL) corren aparte, en un init-container con credencial propia.

## Redis: ACL + montaje inmutable

```
args: [--aclfile, /etc/redis/acl.conf]
volumeMounts:
  - mountPath: /etc/redis/acl.conf
    readOnly: true          # el pod no puede modificar su propia ACL
```

```
$ FLUSHALL    -- como svc_backend
NOPerm User svc_backend has no permissions to run the 'flushall' command
```

## Sealed Secrets

- Un `Secret` de Kubernetes normal es base64, no cifrado: cualquiera con acceso de lectura al repo lo decodifica trivialmente
- Sealed Secrets cifra con la llave pública RSA del controlador antes de comitear, solo el controlador en el clúster tiene la llave privada
- Permite que el repo de infra sea público sin filtrar `CLERK_SECRET_KEY`, contraseñas de BD, etc.

## Cloudflare sobre Port forwarding

- `cloudflared` abre un túnel saliente desde el host: ninguna VM tiene puertos de entrada abiertos a Internet
- Todo el tráfico de navegador es HTTPS forzado en el borde de Cloudflare, mitiga Man-in-the-Middle
- Regla *catch-all* ( `http_status:404` ): un hostname no reconocido ni siquiera llega a Traefik

```
ingress:  
- hostname: "*.tectopia.cl"  
  service: http://10.10.2.218:80  
- service: http_status:404
```

# Defensa contra XSS: Helmet + CSP + sanitización

```
app.use(helmet()) // CSP estricta, remueve X-Powered-By
```

```
script-src 'self' 'wasm-unsafe-eval' https://clerk.tectopia.cl ...  
object-src 'none'  
frame-ancestors 'none'
```

- Sin scripts inline permitidos: el vector clásico de XSS reflejado/almacenado
- `dompurify` sanitiza HTML antes de insertarlo (override de versión segura)
- React escapa por defecto: cero uso de `dangerouslySetInnerHTML` en el código
- Defensa en profundidad: navegador, dato y framework cubren capas distintas

# Cabeceras

| Cabecera                                       | Ataque que mitiga                                                                                        |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| X-Content-Type-Options: nosniff                | MIME-sniffing: el navegador no reinterpreta un archivo como ejecutable si el Content-Type dice otra cosa |
| frame-ancestors 'none' / X-Frame-Options: DENY | Clickjacking: nadie puede embeber el sitio en un iframe invisible para secuestrar clics                  |

frame-src https://challenges.cloudflare.com (Turnstile) es una excepción angosta, no relaja la protección contra clickjacking.

## CORS estricto e inmunidad a CSRF clásico

```
app.use(cors({ origin: corsOrigin }))) // nunca "*"
```

`CORS_ORIGIN` se inyecta por ambiente desde el CI/CD según la rama, nunca un asterisco. La API se autentica con `Authorization: Bearer <token>` (no cookies), así que un sitio malicioso no puede hacer que el navegador adjunte ese header automáticamente.

Clase 01: Disponibilidad, el tercer objetivo de seguridad

## Backups en host físico separado

Velero respalda `buscaviejos-main` (08:30 UTC) y `buscaviejos-infra-persistent` (09:00 UTC) diariamente hacia MinIO en VM4, un host físico distinto al de los demás componentes, con retención rolling de 30 días.

Es la aplicación más directa del tercer objetivo de Clase 01: confidencialidad, integridad, y disponibilidad.



## Extra: KEDA y Harbor

- **KEDA scale-to-zero** en previews: pods inactivos 20 min se apagan, motivado por costo, con un beneficio secundario de seguridad (menos superficie activa cuando nadie usa esa preview)
- **Harbor:** `robot$ci-runner` (cuenta dedicada, no admin) separa push (CI) de pull (CD). Tags inmutables por SHA dan trazabilidad commit-imagen

# Otras buenas prácticas

| Práctica                                           | Qué hace                                                                                                                                                |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| ORM (Sequelize)                                    | Evita inyección SQL por diseño. Incluso las pocas consultas raw usan parámetros bindeados ( <code>replacements</code> ), nunca concatenación de strings |
| Error handler centralizado ( <code>app.ts</code> ) | Captura cualquier error no manejado y responde un mensaje genérico, nunca expone stack traces al cliente                                                |
| Hash de contraseña en pgbouncer                    | <code>scram-sha-256</code> (RFC 7677), desafío-respuesta con sal, no contraseña en texto plano                                                          |
| Accesos no autorizados, auditados                  | Cada <code>401 / 403</code> además dispara <code>audit("authz.denied", ...)</code> , no solo se bloquea, queda registrado                               |
| Validación server-side                             | Reglas (ej. color hex <code>#rrggbb</code> , máx. 50 miembros por grupo) se aplican en el backend, no se confía en el cliente                           |
| Active Session Eviction                            | Banear o expulsar a un usuario fuerza el cierre de todos sus sockets activos al instante ( <code>limitUserSockets</code> , vía Redis, multi-réplica)    |

## Antes de aplicar seguridad: CVEs reales detectadas

**Frontend:** js-cookie · react-router (x2) · vite (x2) · ws (x2) ·  
brace-expansion · js-yaml · dompurify · @babel/core

**Backend:** js-cookie · ws (x2) · qs · uuid

Vulnerabilidades reales en dependencias de terceros: exactamente el riesgo de cadena de suministro que un proyecto moderno con miles de paquetes transitivos no puede evitar del todo, solo detectar y cerrar a tiempo.

## Detalle completo: CVEs corregidas

| Paquete         | CVE / Advisory                 | Repos              |
|-----------------|--------------------------------|--------------------|
| js-cookie       | CVE-2026-46625                 | Frontend + Backend |
| ws              | CVE-2026-48779, CVE-2026-45736 | Frontend + Backend |
| react-router    | CVE-2026-42342, CVE-2026-53663 | Frontend           |
| vite            | CVE-2026-53571, CVE-2026-53632 | Frontend           |
| brace-expansion | CVE-2026-45149                 | Frontend           |
| js-yaml         | CVE-2026-53550                 | Frontend           |
| dompurify       | GHSA-vxr8-fq34-vvx9            | Frontend           |
| @babel/core     | CVE-2026-49356                 | Frontend           |
| qs              | CVE-2026-8723                  | Backend            |
| uuid            | CVE-2026-41907                 | Backend            |

## Cómo se cerraron: gate de CI + hardening de imagen

```
- uses: aquasecurity/trivy-action@v0.36.0  
  with: { exit-code: '1', severity: CRITICAL,HIGH }
```

Trivy bloquea el build si hay CVE crítica o alta: la imagen nunca llega al registro. Además, en el Dockerfile del backend:

```
RUN pnpm install --frozen-lockfile --prod && \  
    rm -rf /root/.cache /usr/local/lib/node_modules/npm \  
        /usr/local/bin/npm /usr/local/bin/npx
```

Origen real: CVE-2026-33671, una vulnerabilidad en npm/corepack empaquetado.

## Lo que sigue abierto (1/3)

- 5 `ClusterRoleBinding` a `cluster-admin` (CD runner, CronJob de limpieza, Traefik, Velero, más el binding estándar del sistema)
- Ningún pod tiene `securityContext`: contenedores corren como root
- El tier `persistent` de Postgres no replica la separación DML/DDDL del tier `main`

## Lo que sigue abierto (2/3): la auditoría también tiene riesgo residual

`auditLog.ts` escribe a stdout, no a un almacenamiento propio. La persistencia depende de que Kubernetes capture los logs del contenedor y Promtail los reenvíe a Loki antes de que el pod rote o se reinicie.

Si ese pipeline falla, o la retención de Loki es corta, el rastro de auditoría de aplicación se pierde. El mecanismo de captura es sólido, la garantía de durabilidad depende de un componente externo no verificado aquí.

## Lo que sigue abierto (3/3): otros

- PII (Personally Identifiable Information) (usernames, IDs de Clerk) en logs de Loki sin anonimizar
- Consola de administración de MinIO expuesta en LAN (puerto 9001)



# Qué cambió respecto de lo prometido

| Ítem              | Qué cambió                                       |
|-------------------|--------------------------------------------------|
| Chat              | Sin E2EE                                         |
| Redis             | sin auth -> password -> ACL                      |
| Modelo de guardia | auditoría verificable en dos niveles (app + k3s) |
| CVEs              | solo push (CI), no pull (Harbor)                 |
| Cloudflare        | túnel + TLS, sin Access                          |

# Incidente real: postmortem del primer día en producción

5 fallas el primer día con 2 réplicas del backend:

- DNS de Clerk
- Secuencias de Postgres
- **Sesión pegajosa de Socket.IO**
- Borrado masivo de mensajes
- Expiración de JWT al reconectar

```
sticky:  
  cookie: { name: io, httpOnly: true, secure: true }
```

## Incidente real: casi-desastre del namespace de infra compartido

- Typo de label -> CronJob borró el namespace de infra compartido
- Postgres + Redis de **34 previews activas** afectados
- Causa real: el CronJob ya tenía `cluster-admin`
- Fix de dos capas: label corregido + guardia de nombre

**La seguridad no juega a favor de la eficiencia.**

## Trabajo futuro

- `securityContext` en todos los Deployments
- RBAC acotado para el CronJob de limpieza
- NetworkPolicy de ingress explícito
- Trivy en todas las ramas no preview
- Rate limiting de Socket.IO
- SBOM (Software Bill of Materials) + firma de imágenes (Cosign)

## Qué aprendimos

- Un solo control nunca alcanza
- **La seguridad no juega a favor de la eficiencia**
- Sin proveedor de nube, no hay capa que absorba tus errores

## Cierre

Eso es Tectopia por dentro. Mostramos lo que funciona bien y también lo que todavía no, porque preferimos eso a aparentar que ya está todo resuelto.

**Gracias. Abrimos preguntas.**